

Term Project Progress Report: $Y = A X W$

COP 5522 · October 5, 2026 · Rolando Veliz, Corey Stewart, Zack Whalen, Kieren Gregory

Project Overview

Our project speeds up the computation $Y = A X W$, where A is a sparse matrix (stored in CSR format) and X and W are dense matrices, using sequential, OpenMP and MPI implementations. The work is motivated by power-law graphs, where a few rows hold many connections and most hold very few, so the work per row is highly uneven.

That unevenness drives most of our design questions: load imbalance, irregular memory access, cache behavior and how well each approach scales. The product can also be computed in two equivalent orders, $(AX)W$ or $A(XW)$, and part of our work is learning which order is faster for which problem shape.

Status Against the Requirements

We have met the sequential requirements and are partway into parallelization: OpenMP methods are explored and ready to apply, and an MPI baseline exists but is not yet validated.

Requirement	Status
Correct sequential implementation	Done. File-driven baseline matches the professor’s interface, and optimized versions were checked against it.
Initial profiling	Done for sequential code (compiler vectorization reports, generated assembly, hardware counters). OpenMP profiling methods are established; sparse-case profiling is next.
Multiple sequential optimizations attempted	Done. Several tried, some kept and some dropped.
Effects of significant optimizations measured	Done on two different machines.
Parallelization begun	In progress. MPI baseline written. OpenMP approach chosen from earlier experiments and ready to apply.

Team Contributions

Member	Primary responsibility	Contribution to date
Rolando Veliz	Sequential	Built the baseline and two optimized versions, tested each optimization, compared the two multiplication orders on two machines.
Corey Stewart	OpenMP	Investigated shared-memory parallel methods on a related dense matrix multiplication assignment and selected which to apply to this project.
Zack Whalen	MPI	Wrote the first MPI version: row-partitioned, correctness-first, same file interface as the sequential code.
Kieren Gregory	Support across all three (joined later)	Designed the benchmarking and analysis plan so the three implementations are compared fairly.

Sequential Work (Rolando)

The sequential effort started with a controlled synthetic benchmark, then moved to the professor’s file-driven interface. The goal was to find out which optimizations actually help rather than assume they do. Three versions are now frozen: a baseline computing $(AX)W$, an optimized $(AX)W$, and an optimized $A(XW)$.

What worked

- Reordering the dense multiplication loops so the inner loop walks memory contiguously. This was the largest portable gain, and the compiler vectorizes it far better.
- Computing $A(XW)$ instead of $(AX)W$ for our current sizes. The sparse step then works on fewer columns, which made it the fastest version on both test machines.

What did not transfer well

- Sparse feature blocking and software prefetching helped on the development machine but were inconsistent on others, so we treat them as machine-specific.
- $A(XW)$ is not always better. In earlier tests with the matrix shapes swapped, the preferred order reversed, so the best choice depends on problem shape.

Results and correctness. Optimized versions run roughly 4 to 6 times faster than the baseline on both a university server and an SDSC Expanse node. The two $(AX)W$ versions give identical output; the reordered $A(XW)$ version differs only at floating-point rounding level (all values within tolerance).

Known limitation. Our test matrix has exactly 16 nonzeros per row, which is not like the irregular graphs that motivate the project. Repeating the sequential tests on an irregular matrix is the next priority.

OpenMP Work (Corey)

Before this project, we studied OpenMP on a related assignment: dense matrix multiplication, including a lower-triangular variant, run on two very different machines (an Intel server and an AMD Bridges-2 node). That work was not on the sparse problem itself, but it taught us which methods are worth carrying over. Its triangular case is especially relevant: the work per row grows steadily, much like the uneven rows in a power-law graph.

Methods explored and what we learned

- **Contiguous loop order, tiling and vectorization.** Gave the biggest single-thread gain without risky compiler flags. It matches Rolando's finding in the sequential work.
- **Scheduling strategy.** Static block scheduling lost badly to imbalance when row costs differed. Dynamic scheduling with the heaviest work handed out first was best, but the same ordering hurt guided scheduling. A rule that works for one schedule can fail for another.
- **Task size and decomposition.** Splitting work into small 2-D pieces helped at small sizes but failed at large ones. Tasks must be small enough to balance but large enough to reuse data.
- **Register blocking.** Helped on one machine and was different on the other, so it is machine-specific.
- **Thread binding and thread-pool warm-up.** Binding helped both machines; warm-up helped only the larger one. Both mostly affect how fairly we measure.
- **Performance counters.** Early readings were misleading because idle threads were spinning, which taught us to confirm every explanation with a controlled experiment.

Order we will apply them to this project, and why

1. **First: dynamic scheduling over CSR rows, with the sequential v16/v17 kernels inside, and thread binding.** Rows are the natural unit of work, uneven row lengths are the central problem, and dynamic scheduling was the clearest winner on uneven work. We will still compare static, dynamic and guided on our real matrices.
2. **Second: task size and heaviest-first ordering for very long rows.** These depend on how skewed the matrix is, so we need the first results to know whether they matter.
3. **Last: register blocking, 2-D decomposition and warm-up.** These were the most machine- and size-dependent. We will try them only if the first two leave measurable gaps.

Throughout, we will use the measurement discipline that worked before: interleave configurations within each round, report medians, and repeat final runs.

MPI Work (Zack)

Zack has written a first MPI version that prioritizes correctness over speed. Each process receives an equal share of the rows of A , every process keeps a full copy of X and W , each computes its rows of the result using $(AX)W$, and rank 0 gathers and writes the output. It uses the same four-file interface as the sequential code, so outputs can be compared directly.

We chose this simple design first because a correct distributed version is needed as a reference before any tuning. Its

weakness is known: equal row counts can mean very unequal work, since rows differ widely in nonzeros. It also does not yet use the A(XW) order that helped sequentially.

The baseline still needs to be checked against the sequential output, and its timing covers distribution, computation and gathering, but not file reading or writing.

Benchmarking and Analysis Plan (Kieren)

Kieren joined recently and is supporting all three efforts by setting up a consistent way to compare them. All implementations will use the same matrices, compiler settings and optimization flags, and important runs will be repeated to reduce noise.

We will report execution time, speedup and parallel efficiency:

$$S_p = T_1 / T_p \quad E_p = S_p / p$$

Here T_1 is the sequential time and T_p is the time with p threads or processes. The comparisons planned are:

- **OpenMP:** thread counts and static, dynamic and guided scheduling, since dynamic scheduling may balance uneven rows better at the cost of overhead.
- **MPI:** process counts (across multiple nodes when possible), and equal-row partitioning versus partitioning balanced by nonzeros.
- **Overall:** sequential versus OpenMP versus MPI, and the point where adding threads or processes stops helping, and why (load imbalance, memory behavior, scheduling overhead or communication).

Areas We Will Develop for the Final Report

We are focusing on a few areas where the explanation matters most. Details will be filled in by the team as results arrive.

Area	Why it matters	Our starting point
Sparsity effects	Our current test matrix is uniform, unlike real graphs.	Build an irregular matrix and repeat the sequential tests first.
Load imbalance	Rows differ widely in work, affecting both OpenMP and MPI.	Compare OpenMP schedules and MPI row-based versus nonzero-balanced partitioning.
Communication and scaling limits	Speedup alone does not explain why scaling stops.	Kieren's benchmarks will separate imbalance, memory behavior, scheduling overhead and MPI communication.
Unsuccessful attempts	Failed optimizations count when we can explain them.	Keep recording what did not help, as in the sequential blocking and prefetching results.
Reproducible code	The final submission needs four runnable programs built with one make.	Proposed mapping: current baseline for project-1, best sequential for project-2, first parallel versions for project-3, tuned parallel for project-4.

Next Steps

1. Create an irregular, power-law style test matrix and re-run the sequential versions to see whether our conclusions still hold. (Rolando, Kieren)
2. Implement the OpenMP version using dynamic scheduling over rows and the optimized sequential kernels, then compare scheduling strategies. (Corey)
3. Validate the MPI baseline against the sequential output, then try nonzero-balanced partitioning and the A(XW) order. (Zack)
4. Run the benchmark plan across sequential, OpenMP and MPI and identify where scaling stops. (Kieren, with all)